

YouTube API Services — Compliance Review Response

Response to the request for “a detailed script or screencast demonstrating the complete workflow of how your internal CLI tool uploads the short educational videos to your YouTube channel using YouTube API services along with end results.”

- **API Client:** internal command-line tool, not distributed, not publicly available
 - **Channel:** Doc Alvers Mathe-Labor (UCBPAYyZs5otrErFEAhmSy3w)
 - **Operator and channel owner:** Dr. Michael R. Alvers — the only user of the tool
 - **Scope requested and used:** <https://www.googleapis.com/auth/youtube.upload> — nothing else
 - **Source:** `videopipeline/youtube-upload.mjs` (CLI) and `videopipeline/lib/youtube.mjs` (API layer), both in the appendix
-

1. What the tool is for

docalvers.de is a free, ad-free collection of more than a hundred interactive mathematics and computer-science labs used in the operator’s own teaching. Each lab gets one short explainer video (60–120 seconds) that shows what the lab does.

Those videos are produced entirely on the operator’s own machine and then published to the operator’s own channel. The CLI tool automates the final step — the upload — so that the title, description and tags do not have to be retyped by hand for every video. **That is the tool’s only purpose. It reads nothing from YouTube, it touches no other channel, and it processes no data belonging to any other person.**

2. Who uses it

One person: the channel owner, on one laptop. The tool is not shipped, not hosted, has no other users, and no user-facing interface beyond the operator’s own terminal. Authorisation therefore uses the OAuth “installed application” flow with the channel owner’s own Google account.

3. The complete workflow

The upload is the last of five steps. Steps 1–4 involve no Google API other than Cloud Text-to-Speech and never touch YouTube.

#	Step	Tool	Result
1	A script is written for the lab (6–8 scenes, spoken text per scene)	text file	narration text
2	The narration is synthesised	Google Cloud TTS via our own backend	one MP3 per scene

#	Step	Tool	Result
3	The lab is driven and recorded headlessly	Playwright + Chrome DevTools screencast	one MP4 per scene
4	Scenes, narration, presenter bubble and outro are cut together	ffmpeg	one finished MP4, 2560×1440, H.264
5	The finished MP4 is uploaded	this tool, YouTube Data API v3	a video on the operator's channel

The title, description and tags for every finished video are maintained in one plain-text file, `videos/youtube-metadata.md`. The CLI reads the section belonging to the file being uploaded, so the wording that ends up on YouTube is exactly the wording reviewed beforehand in that file.

4. The API integration in detail

4.1 Authorisation (once, interactive). `node youtube-upload.mjs --auth` starts a loopback HTTP server on `127.0.0.1:8901`, opens the system browser at Google's consent screen for `.../auth/youtube.upload` with `access_type=offline`, receives the authorisation code on the loopback address, exchanges it at `https://oauth2.googleapis.com/token`, and writes the resulting token to `~/.config/docalvers-videos/token.json` with file mode `0600`. The client secret lives in the same directory, never in the repository — the repository is public.

4.2 Access token refresh (every run). The stored refresh token is exchanged for a fresh access token at `https://oauth2.googleapis.com/token` immediately before the upload. Access tokens are never written to disk.

4.3 Upload (per invocation, one video). A resumable upload session is opened with

```
POST https://www.googleapis.com/upload/youtube/v3/videos?
uploadType=resumable&part=snippet,status
Authorization: Bearer <access token>
X-Upload-Content-Length: <bytes>    X-Upload-Content-Type: video/mp4

{ "snippet": { "title": ..., "description": ..., "tags": [...], "categoryId": "27" },
  "status": { "privacyStatus": "private" } }
```

categoryId 27 is “Education”. The session URL returned in the `Location` header then receives the file in a single `PUT`. The response carries the new video id, which the tool prints as a `https://youtu.be/<id>` link together with the Studio edit link.

4.4 Visibility. The tool defaults to `private`. While this review is pending the API enforces private uploads regardless, and the tool does not attempt to work around that. Videos meant for pupils are set to *unlisted* afterwards, in the web interface.

4.5 Nothing else. The tool calls no other endpoint. It does not read analytics, does not list or modify other videos, does not read comments, does not manage playlists, and never acts on behalf of any account other than the channel owner's.

5. Quota

A single upload costs roughly 1600 quota units. The channel holds eleven videos at the time of writing; the expected rate going forward is **a handful of uploads per month** — one per new lab, invoked by hand. The quota increase was requested so that a batch of finished videos can be published in one afternoon without hitting the daily limit, not to enable automated or high-volume publishing.

6. Source code

Both files are short and are reproduced in full in the appendix of this document, so the review does not depend on repository access. They are also public at

<https://github.com/malvers/mathlab/tree/main/videopipeline> — `youtube-upload.mjs` (command line, argument handling, metadata lookup) and `lib/youtube.mjs` (OAuth and the API calls).

7. A real run, end to end

The workflow below was executed on 25 August 2026. This is the unedited terminal output of the command, not a description of it:

```
$ node youtube-upload.mjs videos/subtraktion-demo-1440p.mp4 --meta --privacy private
→ Uploading: /Users/.../videopipeline/videos/subtraktion-demo-1440p.mp4
  Title:      Schriftliche Subtraktion – das Borgen-Domino | Doc Alvers Mathe-Labor
  Tags:       Mathe, Grundschule, schriftliche Subtraktion, Borgen, Doc Alvers, Unterricht
  Size:       5.7 MB · visibility: private
✓ uploaded: https://youtu.be/bolawEMjhGY
  Studio: https://studio.youtube.com/video/bolawEMjhGY/edit
```

(The video itself is in German — it teaches written subtraction to German pupils. The tool's own output is English.)

End result: video id `bolawEMjhGY` on the channel, carrying exactly the title, description and tags from the metadata file shown above, with `privacyStatus: private`. Being private, it is not publicly retrievable — an `oEmbed` request for it returns HTTP 403 and it does not appear in the channel feed, which is the expected and intended behaviour while this review is pending. We are happy to make it viewable to the reviewing account on request.

The step before it, run once beforehand, was the authorisation:

```
$ node youtube-upload.mjs --auth
✓ access granted – token stored in ~/.config/docalvers-videos/token.json
```

The stored token carries exactly one scope, `https://www.googleapis.com/auth/youtube.upload`, and the file is written with mode `0600`.

For comparison, the dry run of the same command sends nothing and only prints the payload:

```
$ node youtube-upload.mjs videos/subtraktion-demo-1440p.mp4 --meta --dry-run
...
[dry run] nothing was sent.
```

8. Screencast

If the written walkthrough above is not sufficient, we will gladly record a screen capture of the same sequence — terminal, consent screen, upload, and the resulting video in YouTube Studio — and send it. Please let us know; we did not want to delay this response for it.

9. Note on the videos currently on the channel

The videos on the channel today (ten public, one unlisted) were uploaded by hand through the YouTube web interface, because this review is still pending and the tool had not been authorised until now. Going forward the CLI is the intended path for every new lab video.

Contact: Dr. Michael R. Alvers · docalvers.de

Appendix A — youtube-upload.mjs

The command-line entry point: argument handling, metadata lookup, validation.

```
#!/usr/bin/env node
/* Internal CLI that publishes one finished lab demo to Doc Alvers' own YouTube channel.
 *
 * It is the last step of the video pipeline: a lab is recorded headlessly (lib/record-
 * cdp.mjs),
 * narrated (lib/tts.mjs), cut together (lib/assemble.mjs) – and this command hands the
 * finished mp4
 *
 * to the YouTube Data API v3. One video per invocation, always started by hand.
 *
 * node youtube-upload.mjs --auth                    one-time OAuth consent (opens the
 * browser)
 *
 * node youtube-upload.mjs videos/intro-1440p.mp4 --meta [--privacy private] [--dry-run]
 *
 * node youtube-upload.mjs <file> --title "..." --description-file <path> --tags "a,b,c"
 *
 * --meta takes title, description and tags from videos/youtube-metadaten.md, where every
 * finished
 *
 * video already has its section – that file is the single place the wording is maintained.
 *
 * Scope used: https://www.googleapis.com/auth/youtube.upload (upload only, nothing else).
 * Credentials: ~/.config/docalvers-videos/client_secret.json, token cached alongside with
 * mode 0600.
 * Default visibility is "private" – the API forces it anyway until the compliance review is
 * passed.
 */
import fs from 'fs';
import path from 'path';
import { authorize, uploadVideo } from './lib/youtube.mjs';
import { VIDEO_LIB } from './lib/paths.mjs';

const META_FILE = path.join(VIDEO_LIB, 'youtube-metadaten.md');

function parseArgs(argv) {
  const out = { flags: new Set(), opts: {}, file: null };
  for (let i = 0; i < argv.length; i++) {
    const a = argv[i];
    if (a === '--auth' || a === '--meta' || a === '--dry-run') out.flags.add(a.replace(/^--/, ''));
    else if (a.startsWith('--')) out.opts[a.slice(2)] = argv[++i];
    else out.file = a;
  }
  return out;
}

/* One section per video in youtube-metadaten.md:
```

```

    ## <filename> (optional note)
    **Titel:** ...
    **Beschreibung:** ... (may run over several lines)
    **Tags:** a, b, c                                     */
function metaFor(basename) {
  if (!fs.existsSync(META_FILE)) throw new Error('Metadata file missing: ' + META_FILE);
  const text = fs.readFileSync(META_FILE, 'utf8');
  const start = text.split('\n').findIndex((l) => l.startsWith('## ') &&
    l.includes(basename));
  if (start < 0) throw new Error('No section "## ' + basename + '" in ' + META_FILE);
  const lines = text.split('\n').slice(start + 1);
  const end = lines.findIndex((l) => l.startsWith('## '));
  const block = (end < 0 ? lines : lines.slice(0, end));

  const field = (label) => {
    const i = block.findIndex((l) => l.startsWith('**' + label + ':**'));
    if (i < 0) return '';
    const rest = [block[i].slice(('**' + label + ':**').length).trim()];
    for (let j = i + 1; j < block.length && !block[j].startsWith('**'); j++)
      rest.push(block[j]);
    return rest.join('\n').trim();
  };
  const tags = field('Tags');
  return {
    title: field('Titel'),
    description: field('Beschreibung'),
    tags: tags ? tags.split(',').map((t) => t.trim()).filter(Boolean) : [],
  };
}

const args = parseArgs(process.argv.slice(2));

if (args.flags.has('auth')) {
  await authorize();
  console.log('✓ access granted – token stored in ~/.config/docalvers-videos/token.json');
  process.exit(0);
}

if (!args.file) {
  console.error('Usage: node youtube-upload.mjs <video.mp4> --meta [--privacy private] [--dry-run]');
  console.error('          node youtube-upload.mjs --auth          (one time)');
  process.exit(1);
}

const file = path.resolve(args.file);
if (!fs.existsSync(file)) { console.error('File not found: ' + file); process.exit(1); }

```

```

const meta = args.flags.has('meta')
  ? metaFor(path.basename(file))
  : {
    title: args.opts.title || '',
    description: args.opts['description-file']
      ? fs.readFileSync(args.opts['description-file'], 'utf8')
      : (args.opts.description || ''),
    tags: (args.opts.tags || '').split(',').map((t) => t.trim()).filter(Boolean),
  };

if (!meta.title) { console.error('No title - pass either --meta or --title.');
```

```

    process.exit(1); }
if (meta.title.length > 100) { console.error('Title too long (100 characters max).');
```

```

    process.exit(1); }

const privacy = args.opts.privacy || 'private';
if (!['private', 'unlisted', 'public'].includes(privacy)) {
  console.error('--privacy must be private, unlisted or public.');
```

```

  process.exit(1);
}

const id = await uploadVideo(file, { ...meta, privacy, dryRun: args.flags.has('dry-run') });
if (id) console.log('  Studio: https://studio.youtube.com/video/' + id + '/edit');
```

Appendix B — lib/youtube.mjs

The API layer: OAuth consent, token refresh, resumable upload.

```
// YouTube upload for the video pipeline (installed-app OAuth, no npm deps).
// Credentials: ~/.config/docalvers-videos/client_secret.json ; token cached alongside.
// Note: until the Google API audit is passed, API uploads are locked to "private" by
//       YouTube.

import fs from 'fs';
import os from 'os';
import http from 'http';
import { execFileSync } from 'child_process';

const CFG = os.homedir() + '/.config/docalvers-videos';
const { installed } = JSON.parse(fs.readFileSync(`${CFG}/client_secret.json`, 'utf8'));
const TOKEN_FILE = `${CFG}/token.json`;
const SCOPE = 'https://www.googleapis.com/auth/youtube.upload';

export function authUrl(redirect = 'http://127.0.0.1:8901') {
  const q = new URLSearchParams({
    client_id: installed.client_id, redirect_uri: redirect, response_type: 'code',
    scope: SCOPE, access_type: 'offline', prompt: 'consent',
  });
  return `https://accounts.google.com/o/oauth2/v2/auth?${q}`;
}

export async function authorize() { // one-time: opens browser, catches the code
  // locally
  const redirect = 'http://127.0.0.1:8901';
  const code = await new Promise((resolve, reject) => {
    const srv = http.createServer((req, res) => {
      const q = new URL(req.url, redirect).searchParams;
      const c = q.get('code'), err = q.get('error');
      res.end(c ? 'OK - you can close this window.' : (err ? 'Denied: ' + err : 'waiting ...'));
      /* the browser also asks for /favicon.ico - only a request that carries code or error counts */
      if (c) { srv.close(); resolve(c); }
      else if (err) { srv.close(); reject(new Error('OAuth denied: ' + err)); }
    });
    srv.listen(8901, '127.0.0.1', () => execFileSync('open', [authUrl(redirect)]));
  });
  const tok = await (await fetch('https://oauth2.googleapis.com/token', {
    method: 'POST', headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
    body: new URLSearchParams({ code, client_id: installed.client_id, client_secret:
      installed.client_secret,
```

```

        redirect_uri: redirect, grant_type: 'authorization_code' })),
    })).json();
fs.writeFileSync(TOKEN_FILE, JSON.stringify(tok), { mode: 0o600 });
return tok;
}

async function accessToken() {
    const tok = JSON.parse(fs.readFileSync(TOKEN_FILE, 'utf8'));
    const r = await (await fetch('https://oauth2.googleapis.com/token', {
        method: 'POST', headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
        body: new URLSearchParams({ refresh_token: tok.refresh_token, client_id:
            installed.client_id,
            client_secret: installed.client_secret, grant_type: 'refresh_token' })),
    })).json();
    if (!r.access_token) {
        throw new Error('No access token – the refresh token expired or was revoked. Run
            authorize() once. ' +
            JSON.stringify(r).slice(0, 200));
    }
    return r.access_token;
}

export async function uploadVideo(file, { title, description = '', tags = [], privacy =
    'private', dryRun = false }) {
    const size = fs.statSync(file).size;
    console.log(`→ Uploading: ${file}`);
    console.log(`  Title:      ${title}`);
    console.log(`  Tags:        ${tags.join(', ')}`);
    console.log(`  Size:        ${((size / 1e6).toFixed(1))} MB · visibility: ${privacy}`);
    if (dryRun) { console.log(' [dry run] nothing was sent. '); return null; }
    const at = await accessToken();
    const meta = { snippet: { title, description, tags, categoryId: '27' }, status: {
        privacyStatus: privacy } };
    const init = await fetch('https://www.googleapis.com/upload/youtube/v3/videos?
        uploadType=resumable&part=snippet,status', {
        method: 'POST',
        headers: { Authorization: `Bearer ${at}`, 'Content-Type': 'application/json', 'X-Upload-
            Content-Length': size, 'X-Upload-Content-Type': 'video/mp4' },
        body: JSON.stringify(meta),
    });
    const session = init.headers.get('location');
    if (!init.ok || !session) {
        throw new Error('Upload session refused (HTTP ' + init.status + '): ' + (await
            init.text()).slice(0, 300));
    }
    /* whole file in memory – fine for the few hundred MB a demo has, and it keeps the PUT a
       single
       retryable request */
    const put = await fetch(session, { method: 'PUT', headers: { 'Content-Length': size,
        'Content-Type': 'video/mp4' }, body: fs.readFileSync(file) });
    const j = await put.json().catch(() => ({}));

```

```
if (!put.ok || !j.id) {  
    throw new Error('Upload failed (HTTP ' + put.status + '): ' + JSON.stringify(j).slice(0,  
        300));  
}  
console.log(` ✓ uploaded: https://youtu.be/${j.id}`);  
return j.id;  
}
```